面向对象程序设计——初步方案报告

Mandol 智能体记忆—融合索引增强模块

唐菁 2024K8009970006

1 系统理解

1.1 系统目标

让**长期对话**智能体能够有效地**提取、存储和检索**历史信息，从而在用户后续提问时，**找到真正相关的记忆**，并将其提供给上层大模型。

它主要解决了以下三个问题：

1. **提取**：如何从原始对话中提取有价值的信息，构建多种不同形式的记忆。
2. **存储**：如何将这些记忆组织、存储，使系统能够高效管理大量长期记忆。
3. **检索**：如何从大量记忆中找出相关内容，经过筛选、去重、重排和上下文组装等，形成可供大模型使用的记忆上下文。

1.2 基础设计

1.2.1 记忆的三塔结构和分层设计

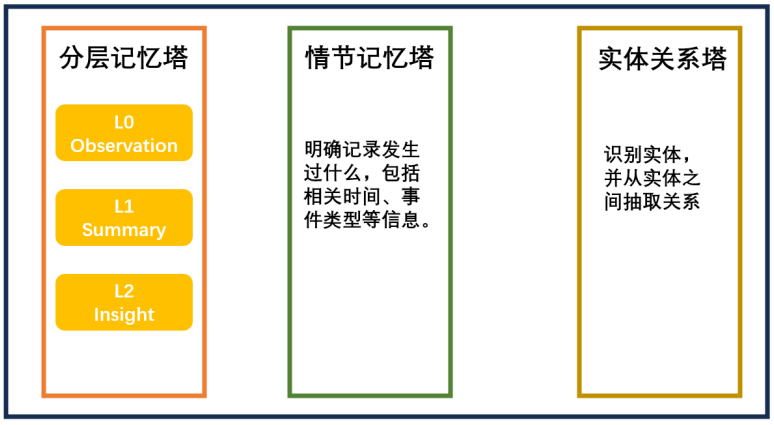


图 1: 记忆的三塔结构示意图

首先，记忆由三塔进行存储。举个例子，“那天他到底怎么说的”属于层层归并的对话记录，属于分层记忆塔；“这个人跟谁、和什么有关”属于实体间的关联网络，属于实体关系塔；“什么时候发生了什么、因为什么”属于带时间脉络的情节记忆塔。

而分层设计沿层级向上压缩：底层保留每一轮对话原话和上下文，中间层把一个会话概括成要点，顶层再汇总成对整段关系的整体理解。

1.2.2 记忆的检索设计

分作以下几步:

- 路由：先决定查哪些塔，而不是一开始就搜索全部记忆
- 范围筛选：每个检索塔只搜索指定的记忆范围, 系统明确每个检索塔可以查询哪些记忆。
- 多路召回: 系统不会只依赖一种匹配方法，而是使用三种互补的检索方式：Cosine 稠密向量检索、BM25 词法检索和 SPLADE 稀疏检索。
- 结果融合：三种检索方式分别返回候选记忆及其分数，利用排名将其统一标准。
- 塔内重排：将查询与候选记忆一起输入模型，直接评估两者之间的相关性。
- 跨塔归并：把不同检索路径找到的内容放到一起

  分层记忆塔可能找到了项目的概括，情节记忆塔找到了相关事件，实体关系图塔则找到了与项目有关的技术信息。系统会将被启用的检索塔返回的候选记忆合并，再对候选分数进行全局归一化
- 进一步筛选：

  如 MAD(中位数绝对偏差) 筛选，排除明显偏低的结果；跨塔消歧，处理同一件事被重复找到的问题；MMR（Maximal Marginal Relevance，最大边际相关性），在有限篇幅中保留尽可能有价值的信息，对候选之间的重复施加惩罚。

绘出设计图如下

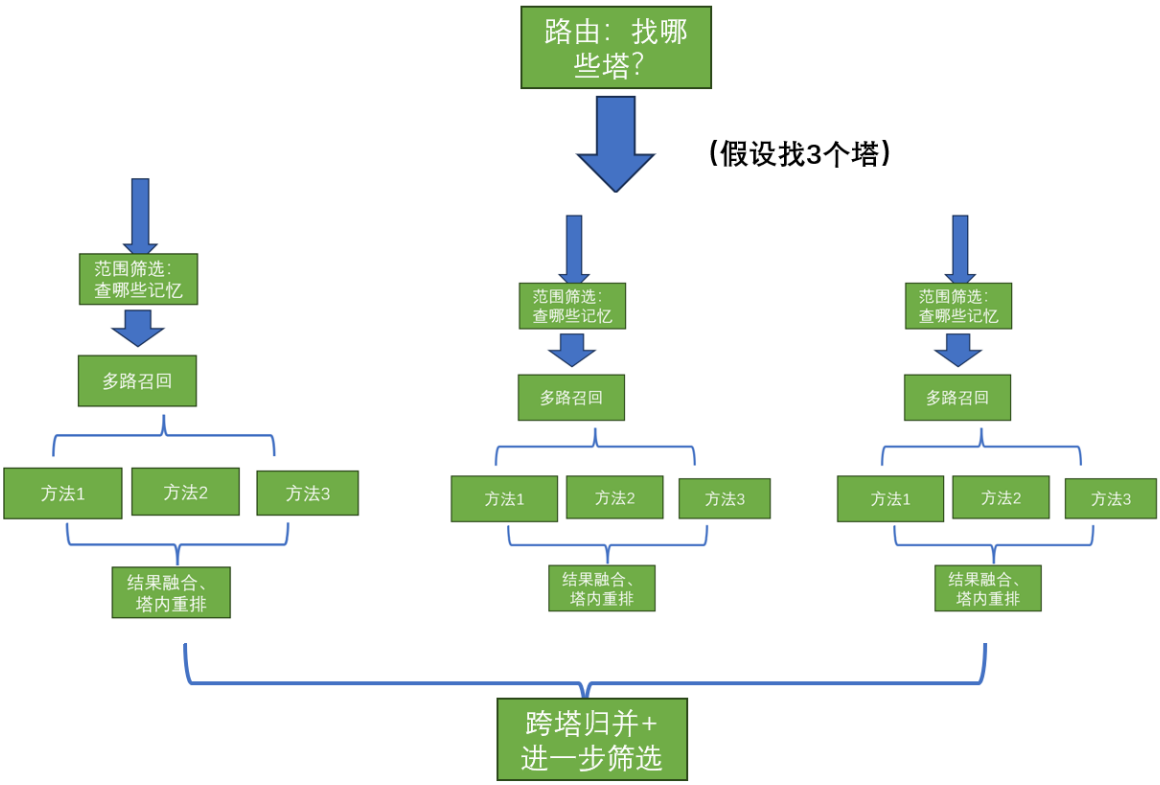


图 2: 记忆的检索设计流程图

1.3 系统对象表及相关关系

1.3.1 存储记忆的对象

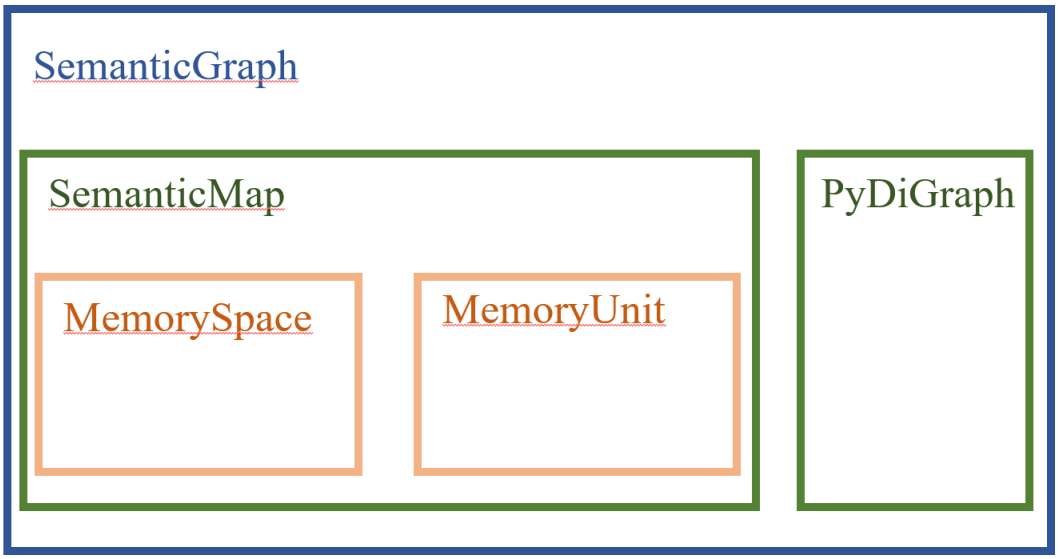


图 3: 存储记忆的对象的基本组成（未包含方法）

以上是存储记忆的对象的基本组成。接下来，将由下至上地阐述各对象是什么、由哪些组成、维护了什么规则、实现了什么方法。

- MemoryUnit

  MemoryUnit 是**一条记忆的值对象**。它包含"uid", "_raw_data", "metadata", "embedding", "sparse_embedding", "text_cached", "created_time" 等属性，记录了记忆的 uid 和内容等

  维护了获取内容、比较相等性、获得稀疏向量等方法。

- MemorySpace

  MemorySpace 是一棵树，表示记忆空间，其有子空间、有记忆成员。它是实现多层记忆、三塔记忆的核心。

  比如分层记忆结构:hierarchical_memory 之下有 3 个子空间，L0_Observation、hierarchical_memory:L1_Summary 和 hierarchical_memory:L2_Insight

  而记忆成员是一个 uid 的字典，记录了各个 MemoryUnit 对象，表示直接属于这个空间的记忆对象。

- SemanticMap

  SemanticMap 是 Mandol 的**核心记忆管理对象**，完成 5 项工作：

  - 记忆存储管理器：保存 MemoryUnit，存储了记忆本体，可以按照 uid 找到每一条记忆；同时管理 MemorySpace，存储了记忆空间，并且维护了它们之间的关系，使其成为一棵树。
  - 索引管理器：实现整数索引 id 和向量之间的查询、转换：可以从整数索引 id 重构出对应的向量表示, 也可以从向量表示重构出整数索引，
  - 检索接口的实现者：继承 RetrievalInterface，提供基于向量的检索能力。
  - 建立多个系统之间的连接，即 id 映射：将记忆的 uid 和整数索引 id 建立双向映射
  - 管理状态：修改与删除记忆等

  举例，当新增一条记忆的时候，SemanticMap 将进行:

  - 处理记忆内容: 生成或获取 embedding，并将相应向量写入记忆对象。
  - 保存记忆: 将该 MemoryUnit 放入 memory_units，通过 UID 管理这条记忆。
  - 维护空间归属: 按照调用方指定的空间，将 UID 加入相应的 MemorySpace。
  - 维护检索索引: 维护 ID 映射、向量索引等。
  - 维护运行状态

可见，这个对象不仅包括了上述的两个对象，还实现了对它们的管理、连接等。

- PyDiGraph

  它是第三方 rustworkx 提供的**有向多重图，用于表示实体之间的关系**。由节点载荷、边载荷、多重边等组成，同时用整数下标寻址。

- SemanticGraph

  它是把**记忆与关系组织成的一个完整的系统**,包括上述提到的两个核心对象:SemanticMap 和 rustworkx.PyDiGraph。

  但是它不止包含了两个对象，还:

  – 维护了成员对象的一些规则、使用方法，比如 Mandol 的图节点可以表示 memory_unit，也可以表示 memory_space；比如图的基本查询、关系建立

  – 维护了成员对象之间的关系，如 uid 和图 index 的双向映射，并保证了二者的同步更新

  – 简化接口：向其他模块提供了底层模块的一些服务，比如想使用 get_unit 来获取记忆单元，不用知道底层是 SemanticMap，只用知道 SemanticGraph 即可调用

1.3.2 记忆的检索的相关对象

将从上至下地阐述记忆检索相关的对象。

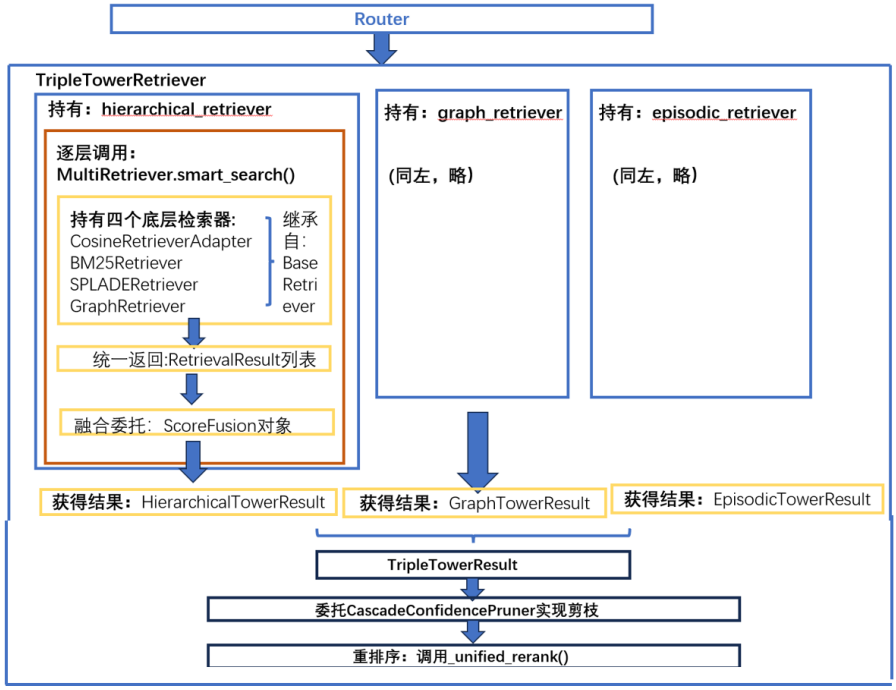


图 4: 记忆检索的相关对象及其关系

首先是 Router, 路由对象 Router，它负责判断本次查询需要哪些塔，生成 TowerRoutingConfig，作为后续检索流程的配置。

然后是记忆检索主体对象：TripleTowerRetriever。

它通过它所持有的 3 个 retriever 对象:hierachical_retriever、graph_retriever 和 episodic_retriever 分别实现对分层记忆、图记忆和情节记忆的检索，获得三类结果对象:HierarchicalTowerResult、GraphTowerResult 和 EpisodicTowerResult，并通过自身所有的方法 _build_candidates() 合并候选，生成 TripleTowerResult 对象，然后委托独立的 CascadeConfidencePruner 对象实现剪枝，最后调用自身所有的方法 _unified_rerank() 实现重排序，获得最终的记忆结果。

其中，hierachical_retriever、graph_retriever 和 episodic_retriever 三个检索对象会委托 MultiRetriever 对象进行搜索，且它们共享同一个 MultiRetriever 实例。这个 MultiRetriver 对象持有 4 个底层检索器，分别是 CosineRetrieverAdapter、BM25Retriever、SPLADERetriever、GraphRetriever，它们都继承自基类 BaseRetriever，并统一返回 RetrivalResult 对象列表，最后再委托 ScoreFusion 对象进行融合，获得该塔的结果。

1.4 记忆的提取的相关对象

由上至下地讲述负责记忆的提取的相关对象。

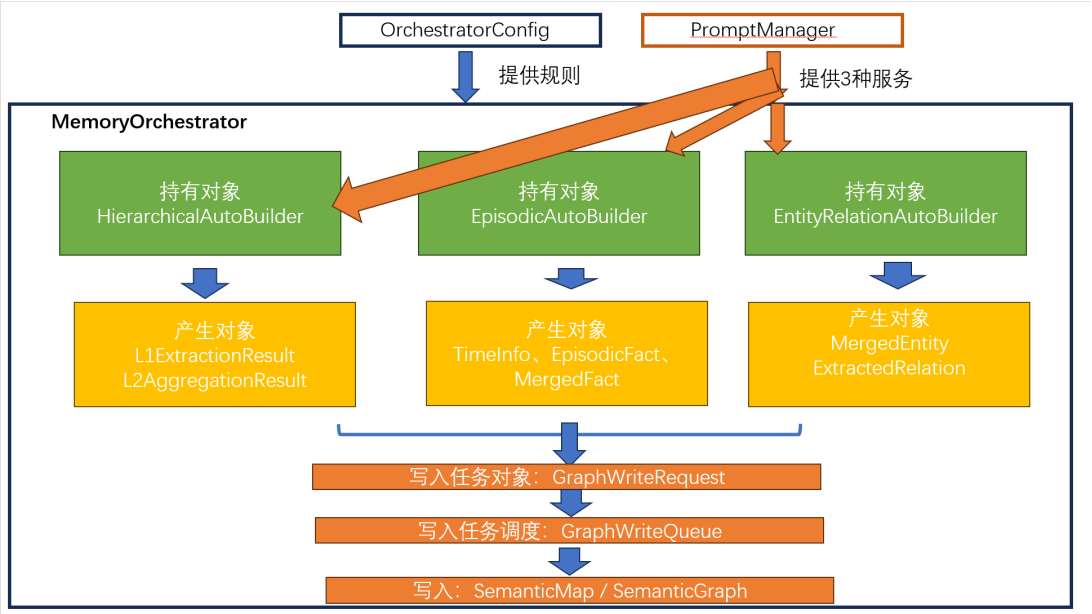


图 5: 记忆提取相关对象关系图

首先，由 OrchestratorConfig 来生成配置对象信息，而后交给 MemoryOrchestrator 进行记忆的提取和写入。

MemoryOrchestrator 持有 3 个对象:HierarchicalAutoBuilder、EpisodicAutoBuilder 和 EntityRelationAutoBuilder，分别负责层级化记忆提取、情节记忆提取和实体关系记忆提取，三者相互独立，由 MemoryOrchestrator 进行协调和使用。

HierarchicalAutoBuilder 负责将 L0 观测转换为更高层次的记忆，通过 extract_l1_from_l0_units 方法生成 L1 记忆，再通过 aggregate_l2_from_l1 方法生成 L2 记忆，分别生成 L1ExtractionResult 列表和 L2AggregationResult 列表。

EpisodicAutoBuilder 负责从 L0 单元中抽取事件或情节事实，串行调用 extract_from_l0_units 方法、deduplicate_facts 方法和 add_to_semantic_system 方法，实现事实抽取、去重，转换为 MemoryUnit 对象并写入

EntityRelationAutoBuilder 负责把对话转换为实体与关系，和 EpisodicAutoBuilder 的方法流程唯一不同的是在最后一个方法前加上了 extract_relations_from_entities，以抽取实体间的关系

这三个对象由 3 种 PromptManager 对象提供 3 种提示词构建服务：HierarchicalPromptManager、EpisodicPromptTemplateManager 和 EntityRelationPromptManager。

然后，由这 3 个 Builder 对象产生的结果被封装成写入任务对象 GraphWriteRequest，并被写入任务调度对象 GraphWriteQueue 进行管理，最后写入前述记忆存储对象：SemanticMap / SemanticGraph

1.5 系统模块及相互组合

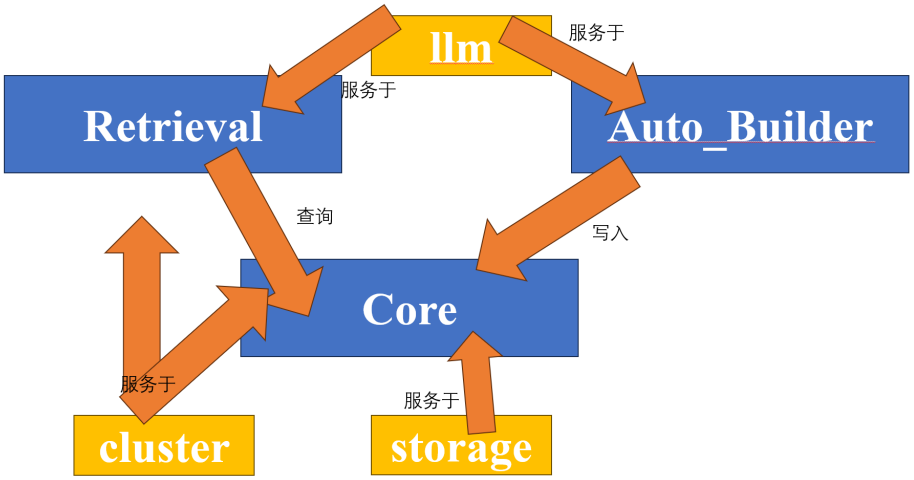


图 6: 系统模块及相互组合（注：utils（模型缓存、配置、日志）是零跨包依赖的最底层未画出）

上述的对象组成了 4 个模块：core、retrieval、auto_builder 和支撑域，实现核心存储、记忆检索、记忆构建和支撑能力。

auto_builder 写入 core；retrieval 是 core 的查询方，core 是检索的数据源和索引载体；支撑域有几个模块:utils 是最底层的设施，如日志等，和所有部分发生关系；llm 给 auto_builder 和 retrieval 提供 LLM 调用；storage 主要服务 core 的存储问题；cluster 是图/向量聚类，服务核心域和分层检索。

1.6 系统实现结果

LoCoMo 长对话基准上的综合答题准确率高达 92.21%，LongMemEval 长期记忆基准上的综合答题准确率高达 88.40%,LongMemEval Token 比 EverMemOS 少 17.4%-20.0% ，且检索平均延迟 Mandol 是 82.2 ms，对比最快非 Mandol 基线 MemOS 是 440.5 ms，加速约 5.4 倍；写入平均延迟，Mandol 是 39.7 ms，而 MemOS 191.9 ms，加速约 4.8 倍。

可见，Mandol 实现了高效率、低成本的记忆提取、存储、检索系统，长对话记忆提取精准、有效。

2 需求陈述

现有系统如果想要通过条件: 属性、时间、图关系来约束记忆收集结果，只能通过改写查询文本，让由词法匹配与向量相似度去猜测条件的含义，这是语义上的“软约束”，没有硬性的条件检索。

因此，提出以下需求陈述：

1. 将记忆检索从仅支持空间成员关系，扩展到可以根据记忆属性、图中关系、语义相似性、时间条件等条件来进行检索，其中时间条件可以是事件时间或记录时间
2. 从仅有析取关系扩展到可以支持任意合取、析取关系
3. 实现查询规划，即对给定的条件组合，系统需能选择候选集的生成方式与执行顺序而非固定单一路径。如当某个条件筛选后剩余候选较少时，先执行该条件
4. 检索效果不低于原基线；维护成本上，尽量不破坏现有接口、召回后端等；并报告访问开销。

3 差距分析

1. 属性约束未完成

   有一个谓词函数 filter_memory_units 支持 eq、ne 和 in 等十种操作符，看似可以按照属性过滤，但是全仓库没有调用它、并且读不了 Metadata，但是许多常用属性如 session_id、confidence、layer 等都在 metadata 里。

2. 时间约束未完成

   时间字段存在语义混乱、字段相互矛盾的现象。如 auto_session_assigner 要算“分钟级间隔”来切会话，它的候选列表是：

   ```
   _EVENT_TIMESTAMP_FIELDS = ("timestamp", "datetime","created_at")
   ```

但 created_at 都是记忆入库的记录时间，而不是对话的时间。这就导致，如果一个 unit 没有 timestamp/datetime，它就会拿到入库的记录时间，导致会话无法被切分。

3. 图的关系约束未完成

   图的关系约束的基本组件，如判断 k 跳、关系类型等均有，但是发生在融合和重排之后，只能做召回后的知识扩展，不能做召回前的候选选择，不参与打分，也不参与条件判断。

4. 合取析取关系未完成

   查询对象 QueryBundle 只有查询文本和三个特征向量，没有字段规定了条件。

5. 查询规划未完成

   FAISS 通道内部，代码中存在查询规划函数 _determine_search_strategy，但是永远返回同一个常量。不过四种过滤执行的方法，即 post-filter、in-filter 、pre-filter 和倒排侧裁剪已经写好了, 只是缺少一个规划函数、选择函数。

   宏观上，也没有进行选择性筛选，没有看某条件能筛掉多少数据。而条件越能筛，候选集越小，就越该先算。

4 扩展定位

需要添加这几个模块:

1. 时间字段的增加

   在最底层的记忆单元 MemoryUnit 加两个字段：event_time：事情什么时候发生,record_time：什么时候入库

   原因：时间属性本身属于一条记忆，一条记忆的最小单元就是 MemoryUnit，所以将时间放入其中合适；如果继续放入 Metadata，filter_memory_units 无法读取。

2. 属性字典的增加

   给一些属性键值 → uid 集合的表格:summary_type、is_structured、fact_type、entity_type 等

   原因：一一读取效率较低；建立表格经计算只需要建立 30 个表格，工程量适度。

3. 图中关系候选实现 GraphHop 求值时复用现成的

   GraphContextExpander.expand_context_by_hops，输出符合相应关系的 uid 集合，作为候选集。

   原因：已有的 expand_context_by_hops 已支持多跳、关系类型过滤等，可以直接复用，作为候选集的过滤。

4. 多种条件约束查询的核心：建立独立文件 query.py，将查询对象和约束树的主体放进其中

   原因：对象天然支持任意嵌套和否定，可以很好地表达与、或、非等逻辑关系；且单独成模块，可以更好地进行测试。如果放入 smart_search 之中，即在这个公共搜索接口加上逻辑关系，一是单一函数难以表达，二是极大地破坏了公共搜索接口的结构。

5. 在 SemanticGraph 加 query(MemoryQuery) 方法，实际搜索在 query 模块之中。下层实现 smart_search 加一个可选的 candidate_uids，即可直接在候选集之中进行搜索。

   原因；实现“方法转发”，从而不用知晓 query 的具体实现，亦可调用条件搜索；条件筛选对于下层搜索的影响仅影响候选集，而不影响其逻辑，可维护性更强。

6. 实现选择性，即这个条件能筛掉多少数据。条件越能筛，候选集越小，就越该先算。这必须同时看到全部约束和索引统计。

   这相对复杂，设计独立 Planner.py 层，不嵌在老组件里。

7. 实现 FAISS 向量的搜索方法选择：SemanticMap 的 _determine_search_strategy 中

   原因：复用作者已写函数。

8. 增加适配器：剪枝器需要的对象是 EnhancedCandidateChunk，但是新的设计产生的不是这个格式，需要增加适配器。

   原因：不修改已有剪枝器，尽可能减少对原来代码的影响，而是让新格式写适配器适应老格式。

5 对象建模

将分别按照流程介绍各层对象，并介绍各层内对象之间的相互关系。

1. “门面”层

   指 SemanticGraph.query() 作为入口，执行 QueryEngine 对象中的 execute 方法。

2. 核心层:QueryEngine

   QueryEngine 是整个任务的总调度，编排任务、给其他对象委派任务。

   编排的任务有：

   建立上下文：委派给 QueryContext，

   做规划：委派给 QueryPlanner

执行计划：委派给 PlanExecutor

结果适配：委派给 PruneAdapter

3. 建立上下文: QueryContext

   QueryContext 持有两种对象，一个是 AttributeIndex，另一个是 TimeIndex。前者实现上文提到的属性值->int_id 集合以方便属性查找，后者实现 (timestamp, int_id) 有序数组，从而实现二分查找满足时间要求的记忆, 提高效率。

4. 做规划:QueryPlanner

   它委派 SelectivityEstimator 估算一个条件大概能筛掉全集的多大比例、根据比例进行排序，然后输出 ExecutionPlan 对象，即执行的计划。

5. 执行计划：PlanExecutor

   根据上文输出的 ExecutionPlan、QueryContext 还有初始的 MemoryQuery，PlanExecutor 按计划开始筛选候选集。它委派 5 个具体的搜索算子，实行两段式搜索：

   第一段是集合段，按照新增的三类条件进行搜索，获得候选集对象 CandidateSet：AttributeFilterOperator 查属性索引，并与当前候选集求交集；TimeFilterOperator 按指定时间轴（事件时间 / 记录时间）查时间索引，并与当前候选集求交；GraphExpandOperator 按指定跳数和关系类型扩展邻居，得到图邻居候选集，再与当前候选集求交。

   其中，由 CandidateSet 对象是集合本身，实现交、并、补、取反等具体方法；ConstraintSet 是条件输入，是条件树的表示，内部结构是一颗递归树。

   第二段是打分段，在 CandidateSet 内部进行打分，是原有的功能，即 VectorSearchOperator 在剩余候选集内做稠密向量检索并打分；LexicalSearchOperator 在剩余候选集内做 BM25/SPLADE 等词法检索并打分，最后输出候选列表，即 ScoredCandidate 对象的列表。

6. 结果适配：PruneAdapter

   为和原有剪枝层对输入对象的要求适配，引入适配层。这样新层依赖既有剪枝器，已经写好的剪枝器不做修改、不反向依赖新层。

6 初步方案

以下讲述数据获取、开发流程及每一步通过测试、以及总体的评测。

数据获取：公开问答数据集 LoCoMo 和 LongMemEval 可免费下载，但公开拿不到经过抽取的三层塔记忆，将尝试发邮件询问负责人。同时，因为需要属性条件筛选，但属性往往藏在 raw_data 或 meta_data 中，需要回填。且此是问答数据集而不是条件查询评测集，所以条件查询评测集需要另外构造，即“时间约束 + 属性约束 + 关系约束”得到相关记忆的数据集。

然后是实现的每一步及评测：

1. 检查数据里属性的覆盖率，并将未覆盖的属性从 raw_data、meta_data 里进行回填 (因为 LLM 已经把对话里的属性抽出来，写进了 raw_data / metadata。现在只是把这些结果搬个位置、统一格式)

2. 建立上下文: QueryContext

   然后进行测试：

   根据新建立的索引查询得到的结果和直接暴力扫描得到的结果相同；索引重建后相应结果也应该更新；同时测试一些边界条件，如空集、全集、没有该属性等情况

3. CandidateSet 的集合运算实现

   测试：集合运算的正确性，以及嵌套运算的正确性等

4. AttributeFilterOperator、TimeFilterOperator、VectorSearchOperator 和 GraphExpandOperator 四个算子的实现

   测试：每个算子的计算和在整个集合上暴力计算的结果相同，并计算每个算子的时间延迟

5. 实现规划层:QueryPlanner

   测试：测试顺序确保按照条件筛选的比例升序排列；plan.describe() 能复现” 为什么走这条路；不同顺序得到的集合结果和固定顺序的集合结果相同

6. 实现适配器 PruneAdapter、将 query 接入 SemanticGraph, smart_search 中接入 candidate_uids，从而从上至下地将已实现的核心部件连接起来。

   实行端到端正确性测试：graph.query(MemoryQuery) 能跑通

   然后是总体的验收评测方案:

1. 构造条件查询评测集，即“时间约束 + 属性约束 + 关系约束”得到相关记忆的数据集。

2. 在原系统上评测原来数据集和条件查询评测集，前者上评测 QA accuracy，后者上测试 Recall@k / nDCG/P@K 等指标

3. 在新系统上评测原来数据集和条件查询评测集，要求前者的结果大于等于原系统的能力，后者的结果相对原系统得到提升，且达到一个可接受下限（如 60%）

注：原系统上进行条件查询评测时，把条件写成自然语言拼进 query，再调用原系统的 smart_search 进行测评。

7 大模型辅助与验证修正

了解整个系统的辅助模型是 deepseek harness，以下列出笔者和模型交互过程中，我对模型的纠正和交互:

1. 模型认为，不同塔之间有不同的属性是错误的、是属性覆盖不全的情况，应该给不同塔补全后再进行属性筛选，如 fact_type 属性只存在于情节塔之中，entity_type 属性只存在于图关系塔之中。

2. 按照模型说的 EnhancedCandidateChunk.timestamp 沿着列表:"timestamp", "event_time", "time", "date", "created", "created_at" 逐个进行尝试。但是阅读代码后，发现前面几个字段都是不存在的，最终装入的是入库时间:created。

3. 在“扩展定位”一节，模型会对各个模块的增加位置提出 2-4 种建议，笔者根据思考、和其他 LLM 对话，进行决策